SalesGenie AI — Development and Repository Setup

Status: Setup proposal for review; not yet run against the application source
Prepared: 15 October 2026
Selected baseline: Python + FastAPI + Streamlit + SQLAlchemy + Alembic; SQLite for local development, PostgreSQL for production; OpenAI API

> The requester supplied `https://github.com/230701224/sales-genie-ai.git` as the repository remote. The source folder was not supplied and this workspace currently contains project reference files only. These instructions are not evidence that the repository, dependencies, Docker, or application have been set up or validated.

1. Prerequisites

- Python 3.10 or newer (training-guide minimum); use the same tested minor version across the team.
- Git.
- Docker Desktop when running PostgreSQL locally.
- Access to the supplied GitHub repository.
- An OpenAI API key for AI-backed development. Obtain it through the team's approved secret-sharing process; never put a real key in Git or in shared documentation.

2. Repository and branch workflow

After the project lead confirms the actual source folder and repository access:

```
git clone https://github.com/230701224/sales-genie-ai.git
Set-Location .\sales-genie-ai
git fetch origin
git branch --all
```

Proposed branch policy:

- `main`: reviewed, demoable baseline; protect it from direct pushes.
- `develop`: shared integration branch, if the team agrees to use it.
- `feature/<ticket>-<short-name>`: short-lived task branches.
- Submit changes through pull requests; require at least one teammate review and passing tests before merging.
- Do not commit `.env`, local databases containing real data, virtual environments, caches, or credentials.

Before creating `develop`, check `git branch --all` and coordinate with repository owners to avoid creating a conflicting branch or overwriting existing work.

3. Windows virtual environment

From the confirmed project root in PowerShell:

```
python --version
git --version
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks activation, use the team's approved execution policy or invoke the environment's Python directly:

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

The repository should provide `requirements.txt` (or a confirmed lockfile), `.env.example`, and `.gitignore`. Do not invent dependency pins until the source tree and tested versions are reviewed.

4. Environment configuration

Copy the example file locally and fill only local values:

```
Copy-Item .env.example .env
```

The eventual `.env.example` should document placeholders such as:

```
DATABASE_URL=sqlite:///./salesgenie.db
OPENAI_API_KEY=
OPENAI_MODEL=
API_BASE_URL=http://127.0.0.1:8000
```

Use a PostgreSQL `DATABASE_URL` for production configuration and for local parity tests when required. Keep actual passwords and API keys out of `.env.example`, documentation, terminal recordings, screenshots, and commits.

5. Local services

The existing guides/screenshots indicate a FastAPI backend on port `8000` and a Streamlit UI on port `8501`. Confirm module paths in the actual repository before running:

```
python -m uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

In a second PowerShell window, activate the same virtual environment and run:

```
python -m streamlit run app.py --server.address 127.0.0.1 --server.port 8501
```

Expected local endpoints after implementation:

- API: `http://127.0.0.1:8000`
- API documentation: `http://127.0.0.1:8000/docs`
- Streamlit: `http://127.0.0.1:8501`

These commands are a proposal based on the supplied guides and screenshots, not verified commands for the unavailable source tree.

6. Database and Docker baseline

- Local development default: SQLite, selected from the training-guide baseline.
- Production target: PostgreSQL.
- Use SQLAlchemy for persistence and Alembic migrations for schema changes.
- Before implementation, decide whether the shared development environment will use SQLite only or also run PostgreSQL in Docker for parity testing.
- The repository should include a reviewed `compose.yaml` for PostgreSQL if Docker is chosen. It should bind the database to localhost, use a non-production local credential supplied through an untracked `.env`, and persist data in a named volume. Do not publish a production password or an API key in the compose file.

No Docker configuration is created or started by this documentation-only change.

7. Initial verification checklist

Once source and environment are available, the Database/DevOps Engineer and Backend Developer should verify:

- A clean clone installs dependencies without manual package fixes.
- The backend starts and `/docs` loads.
- The Streamlit UI starts and reaches the local API.
- A migration applies to SQLite; PostgreSQL migration parity is tested where supported.
- Synthetic seed data loads repeatably.
- Lead CRUD, invalid-input errors, missing-ID behavior, CSV duplicate reporting, and saved-analysis retrieval pass.

- Tests and setup instructions do not require a real customer record or a committed secret.

Record Python, dependency, database, and Docker versions in the repository once the team tests and approves them.